

Grounding DINO Reproduction for Open-Vocabulary Object Detection and Visual Grounding

Group ID: TBD
Collaborators: TBD

Group ID: TBD

Collaborators: TBD

Contribution of each member: TBD

1 Abstract

This project reproduces and evaluates Grounding DINO for two related vision-language tasks: open-vocabulary object detection (OVD) and visual grounding (VG). We use the official Grounding DINO Swin-T OGC checkpoint and evaluate the model in a zero-shot setting without COCO or RefCOCO fine-tuning. The project includes an end-to-end inference pipeline, COCO and RefCOCO-family evaluation, prompt and threshold analysis, protocol checks, failure analysis, and engineering validation of the CUDA extension.

Our self-implemented OVD pipeline achieves **46.16 mAP** on COCO val2017, while the official Grounding DINO COCO evaluation script achieves **48.50 mAP**. For visual grounding, our zero-shot Acc@0.5 results are **50.85%** on RefCOCO val, **45.38%** on RefCOCO testB, **51.67%** on RefCOCO+ val, **46.45%** on RefCOCO+ testB, and **60.95%** on RefCOCOg val. These VG results match or slightly exceed the zero-shot numbers reported by Grounding DINO. We also show that compiling the custom CUDA operators improves inference speed but does not explain the remaining OVD accuracy gap, which is mainly due to differences in post-processing and evaluation protocol.

2 1. Introduction

Open-vocabulary object detection aims to localize objects described by arbitrary text instead of a fixed label set. Visual grounding is a related task where the model localizes the image region described by a natural-language expression, such as "the red car on the left" or "the person holding an umbrella". Both tasks require visual recognition, language understanding, and alignment between image regions and text tokens.

The goal of this project is not to train a new foundation model from scratch. Instead, we focus on reproducing a strong existing open-source model, evaluating it on public datasets, and analyzing where the reproduction matches or differs from the paper. This is a realistic setting for modern computer vision research, where reproducing a large pretrained vision-language model requires careful handling of checkpoints, prompts, post-processing, datasets, and evaluation protocols.

We chose Grounding DINO because it supports both OVD and VG through a unified text-conditioned detection model. The project covers the three required components:

1. Method reproduction: implement a reproducible Grounding DINO inference and evaluation pipeline.
2. Dataset evaluation: evaluate on COCO val2017 and RefCOCO / RefCOCO+ / RefCOCOg.
3. Analysis: perform prompt ablation, threshold sweep, protocol comparison, CUDA extension validation, and failure analysis.

3 2. Related Work

DINO improves DETR-style object detection with denoising anchor boxes and stronger query initialization. Grounding DINO builds on this detection framework and extends it with language-conditioned detection.

Grounding DINO combines a Swin Transformer visual backbone with a BERT text encoder and cross-modal fusion modules. It can detect objects using text prompts and localize referring expressions without requiring a fixed closed-set classifier.

COCO is a standard benchmark for object detection. We use COCO val2017 and the official COCO API / pycocotools evaluation protocol to measure mAP.

RefCOCO, RefCOCO+, and RefCOCOg are standard visual grounding datasets built on COCO images. They evaluate whether a model can localize a region described by a referring expression. We compare our $\text{Acc}@0.5$ results to the zero-shot $@1$ results reported by Grounding DINO, not to the fine-tuned RefCOCO numbers.

Other relevant open-vocabulary or vision-language detection systems include GLIP, OWL-ViT, YOLO-World, and Detic. We use Grounding DINO because it is directly designed for both open-set detection and grounding, and because official checkpoints and evaluation scripts are publicly available.

4 3. Approach

4.1 3.1 Model and Pipeline

We reproduce Grounding DINO with the official Swin-T OGC checkpoint `groundingdino_swint_ogc.pth`. The pipeline has two tasks:

Task	Input	Output	Metric
OVD	Image + COCO category prompt	Boxes, scores, COCO category IDs	COCO mAP
VG	Image + referring expression	Top predicted box	$\text{Acc}@0.5$

The model pipeline is:

1. Encode the image with the Swin-T visual backbone.
2. Encode the text prompt or referring expression with BERT.

3. Fuse image and text features with the Grounding DINO cross-modal transformer.
4. Decode candidate boxes and token-aligned phrase scores.
5. Convert model outputs to task-specific predictions.

For OVD, we concatenate the 80 COCO class names into a single prompt and use token-span based post-processing to map model outputs back to COCO category IDs. For VG, we use the expression as the text query and select the best candidate box according to the semantic matching score.

4.2 3.2 Implementation

The project includes a unified wrapper that supports both the official GroundingDINO source backend and a HuggingFace fallback backend:

Backend	Role	Final usage
gdino	Official GroundingDINO source + OGC checkpoint	Main result backend
hf	HuggingFace IDEA-Research/grounding-dino-tiny	Fallback and early validation

Main implementation components:

- `src/model_wrapper.py`: model loading and unified inference interface.
- `src/ovd/eval_coco.py`: COCO OVD evaluation pipeline.
- `scripts/run_vg_eval.py`: RefCOCO-family VG evaluation.
- `src/ovd/official_postprocess.py`: token-span mapping for OVD.
- `src/vg/box_selection.py`: visual grounding box selection strategies.

4.3 3.3 Datasets and Evaluation Setup

Task	Dataset	Images	Annotation	Scale
OVD	COCO val2017	<code>data/coco/val2017</code>	<code>instances_val2017</code>	5000 images
VG	RefCOCO	COCO train2014	HF parquet / Ref-COCO annotations	val, testB
VG	RefCOCO+	COCO train2014	HF parquet / RefCOCO+ annotations	val, testB
VG	RefCOCOg	COCO train2014	HF parquet / Ref-COCOg annotations	val

Important settings:

Item	Setting
Model	Grounding DINO Swin-T OGC
Checkpoint	weights/groundingdino_swint_ogc.pth
OVD prompt mode	concat_token
OVD prompt template	{name}
OVD best threshold	box=0.15, text=0.05
VG selection	semantic
VG threshold	box=0.05, text=0.05
VG metric	Acc@0.5

We do not fine-tune the model on RefCOCO. Therefore, our VG results should be compared to Grounding DINO’s zero-shot @1 results, not to the much higher fine-tuned RefCOCO results.

5 4. Experimental Results

5.1 4.1 Open-Vocabulary Detection on COCO

Our main OVD result is from results/exp_2026-05-23_ovd_aligned/metrics.json.

Metric	Result
mAP	46.16
AP50	61.20
AP75	50.39
AP_s	31.26
AP_m	49.08
AP_l	60.72
AR@1	35.88
AR@10	58.39
AR@100	61.88

Comparison with the paper and the official evaluation script:

Method	COCO val2017 mAP	Notes
Grounding DINO paper	48.4	Reported zero-shot result
Official test_ap_on_coco.py	48.50	Reproduced in this project

Our self-implemented pipeline	46.16	Best full-run result after threshold sweep
-------------------------------	--------------	--

The official script result confirms that the checkpoint and dataset are correct. The remaining gap in our self-implemented pipeline is most likely caused by differences in post-processing, token-to-category mapping, and evaluation protocol details.

5.2 4.2 Visual Grounding on RefCOCO-family Datasets

The main VG results are from `results/refcoco_gdino/metrics.json`.

Dataset / split	Our Acc@0.5	Hits / Total	Paper zero-shot @1	Difference
RefCOCO val	50.85%	5509 / 10834	50.41%	+0.44
RefCOCO testB	45.38%	2312 / 5095	43.21%	+2.17
RefCOCO+ val	51.67%	5559 / 10758	51.40%	+0.27
RefCOCO+ testB	46.45%	2271 / 4889	45.81%	+0.64
RefCOCOg val	60.95%	2984 / 4896	60.42%	+0.53

These results show that the reproduced VG pipeline is well aligned with the paper’s zero-shot setting. All five evaluated splits match or slightly exceed the reported zero-shot @1 numbers.

5.3 4.3 Prompt Ablation

Prompt ablation results are from `results/coco/ablation_prompt.json`.

Prompt template	mAP	AP50	Number of predictions	Interpretation
{name}	47.05	58.80	2731	Direct class names
a {name}	17.56	24.21	1675	Worse
a photo of a {name}	5.24	6.33	185	Much worse

For COCO OVD, 80 class names are concatenated into one long prompt. Longer templates consume the BERT token budget and cause later categories to be truncated or poorly aligned. Direct class names therefore work best in the concat-token setting.

5.4 4.4 Threshold Sweep and VG Protocol Check

The best VG threshold on a 500-sample subset is:

box_threshold	text_threshold	Acc@0.5	Samples
0.05	0.05	52.20%	500

The VG box selection protocol was also checked on RefCOCO validation with 500 samples:

Strategy	Acc@1	Acc@5	Acc@10
semantic	52.20%	89.60%	95.60%
argmax_official	52.60%	89.60%	95.60%

The top-1 difference is only 0.4 percentage points, and top-5 / top-10 are identical. This suggests that VG performance is mainly controlled by candidate box quality rather than this specific selection rule.

5.5 4.5 Engineering Validation on RTX 5090

On an RTX 5090 server, the GroundingDINO CUDA extension initially failed to compile because the original extension used older PyTorch C++ APIs such as `tensor.type()` and `tensor.data<T>()`. After patching the extension to use PyTorch 2.x compatible APIs, `groundingdino._C` imported successfully and the full OVD evaluation ran with the custom CUDA operator.

Experiment	mAP	Speed observation	Role in report
5090 fallback	44.60	About 8.8 images/s	Engineering check only
5090 CUDA fixed	44.60	About 10.7 images/s	Engineering check only

The CUDA extension improves speed but does not materially change mAP. Therefore, CUDA fallback is not the main explanation for the gap between 46.16 and 48.50. The likely cause remains post-processing and protocol mismatch.

6 5. Limitations and Future Work

6.1 5.1 OVD Limitations

The self-implemented OVD pipeline is still about 2.2 mAP below the paper and about 2.34 mAP below the official script. Since the official script reaches 48.50 mAP using the same checkpoint and dataset, this gap is not caused by the model weights or COCO data. It is most likely due to differences in token-span processing, category mapping, thresholding, or other post-processing details.

Small-object detection is also weaker than medium and large objects. AP_s is 31.26, compared with AP_m 49.08 and AP_l 60.72. This is expected because small objects have weaker visual features after downsampling and are harder to align with open-vocabulary text queries.

6.2 5.2 VG Limitations

The VG evaluation is zero-shot only. It does not reproduce the paper’s fine-tuned RefCOCO results in the 81-89% range. Reproducing those numbers would require supervised RefCOCO training, checkpoint management, and additional compute resources.

We also evaluated RefCOCO testA as a supplementary run on the 5090 server, obtaining $\text{Acc}@0.5$ **45.42%**. This result is useful as additional evidence but is not included in the main comparison table because the main paper-aligned comparison in this report focuses on val/testB/refcocog val zero-shot references.

6.3 5.3 Future Work

If we continued this project, the most useful next steps would be:

1. Fully match the official COCO post-processing and category mapping in the self-implemented OVD pipeline.
2. Add LVIS or ODinW evaluation to test rarer open-vocabulary categories.
3. Implement RefCOCO fine-tuning to compare with the paper’s supervised RefCOCO results.
4. Improve qualitative visualization and failure clustering for small objects, dense scenes, and ambiguous referring expressions.

7 6. Conclusion

This project successfully reproduces Grounding DINO for both open-vocabulary object detection and visual grounding. The OVD self-implemented pipeline reaches **46.16 mAP** on COCO val2017, and the official COCO script reaches **48.50 mAP**, matching the expected paper-level performance. For visual grounding, the reproduced zero-shot pipeline matches or slightly exceeds the paper’s zero-shot @1 results across five RefCOCO-family splits.

The project also demonstrates important practical lessons: prompt format strongly affects OVD, VG thresholding should preserve candidate boxes, CUDA extensions mainly affect speed, and evaluation protocol details can create measurable differences even when the model checkpoint and dataset are correct. Overall, the project satisfies the reproduction, dataset evaluation, and analysis goals of the final project.

8 7. Member Contributions

Member	Contribution
TBD	TBD

9 References

1. Shilong Liu et al. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." ECCV 2024.
2. Hao Zhang et al. "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection." ICLR 2023.
3. Tsung-Yi Lin et al. "Microsoft COCO: Common Objects in Context." ECCV 2014.
4. Licheng Yu et al. "Modeling Context in Referring Expressions." ECCV 2016.
5. COCO API: <https://github.com/cocodataset/cocoapi>
6. pycocotools: <https://github.com/ppwwyyxx/cocoapi>
7. GroundingDINO official repository: <https://github.com/IDEA-Research/GroundingDINO>
8. HuggingFace OWL-ViT documentation: https://huggingface.co/docs/transformers/model_doc/owlvit
9. GLIP repository: <https://github.com/microsoft/GLIP>
10. YOLO-World repository: <https://github.com/AILab-CVC/YOLO-World>
11. Detic repository: <https://github.com/facebookresearch/Detic>

10 Appendix: Reproducibility and Result Paths

10.1 Core Commands

Environment check:

```
python scripts/setup_env.py
```

OVD subset smoke test:

```
python src/ovd/eval_coco.py --config configs/coco_ovd.yaml --subset 20
```

OVD full evaluation:

```
python src/ovd/eval_coco.py --config configs/coco_ovd.yaml
```

VG subset smoke test:

```
python scripts/run_vg_eval.py \  
  --dataset refcoco \  
  --split validation \  
  --max-samples 20 \  
  --hf-dir data/refcoco_hf
```

Official OVD baseline:

```
python scripts/run_official_ovd_baseline.py \  
  --out-dir results/exp_2026-06-15_official_ovd \  
  --num-workers 4
```


10.2 Result Index

Content	Path
OVD self-implemented main result	results/exp_2026-05-23_ovd_aligned/metrics.json
OVD official comparison	results/exp_2026-06-15_official_ovd/comparison.json
VG full main results	results/refcoco_gdino/metrics.json
Prompt ablation	results/coco/ablation_prompt.json
VG threshold sweep	results/exp_2026-05-23_vg_sweep/best_subset.json
VG protocol comparison	results/exp_2026-06-15_vg_protocol/protocol_comparison.json
CUDA build record	results/exp_2026-06-15_cuda_built/status.json
5090 OVD CUDA fixed validation	results_5090_2026-06-18_cuda_fixed_ovd.tar.gz
5090 VG testA supplementary result	results_5090_2026-06-18.tar.gz